DEMO 02 · PREDICTIVE MAINTENANCE

See the Failure Before It Bills You

A ten-pump fleet monitor that projects remaining useful life and raises a maintenance advisory only for the asset that actually needs it.

ADVISORY *Recommends only — no autonomous writes*

AUTHOR Rakovskyi Dmytro · VERSION Full-run demo 02 · DATE 2026 · REPOSITORY github.com/rakovpublic/jneopallium

Unplanned downtime is the expensive failure mode, and the usual defences are both wrong: maintain everything on a fixed calendar and you waste money on healthy machines; wait for things to break and you pay in outage. This demo watches a fleet of ten pumps over a long run, tracks each one's vibration and bearing temperature, and projects remaining useful life — then it stays quiet on the healthy pumps and raises an advisory only for the one that is degrading.

What it is

Demo 02 models predictive maintenance over an MQTT/Sparkplug-style telemetry feed from ten pumps. Its safety ceiling is **ADVISORY**: it recommends, and a planner decides — there is no autonomous write back to any asset. The network has five layers (sized 5·4·3·2·3), enough to turn raw telemetry into a per-asset health estimate and a targeted recommendation.

How it is wired

Figure *Five layers, from raw telemetry to a targeted advisory with an audit reason.*

Signals are typed end to end: `VibrationSignal` and `BearingTempSignal` carry the raw telemetry, `DeviceEventSignal` carries online/offline transitions, `DegradationSignal` and `RemainingUsefulLifeSignal` carry the health estimate, and `MaintenanceAdvisorySignal` carries the recommendation.

What it does

Across the fleet, three patterns appear and the network treats each correctly.

- A degrading pump (pump-03) shows a rising vibration trend; its projected remaining useful life falls sharply and it earns a maintenance advisory.
- A healthy pump (pump-00) holds steady; its remaining useful life stays high and it receives no advisory in the same window.
- A pump that drops offline (pump-07, after tick 8) produces a device-offline advisory rather than being silently ignored.

> **NOTE**
> The discipline that matters: the healthy pump gets zero advisories in the same window the degrading pump gets one. The value is not in flagging everything — it is in flagging the right asset.

The evidence

The demo runs deterministically for a thousand ticks, long enough for a slow degradation trend to separate from baseline noise.

VERIFICATION EVIDENCE	
Ticks	1000 / 1000
Degrading pump-03	remaining useful life **40.0**, advisory raised
Healthy pump-00	remaining useful life **950.0**, no advisory
Offline pump-07	device-offline advisory after t8
Verdict	**PASS**

How to run it

1 Build

Build the worker and the demo model JAR.

```
mvn -q -pl worker -am package
```

2 Run via the framework entry point

Entry loads the model JAR and the demo context through the real local path.

```
java -cp demo-model.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///path/to/demo-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  context.json
```

3 Inspect

Read the per-tick advisories and audit reasons.

```
ls target/jneopallium-fullrun-demos/demo-02-pump-maintenance/
```

Why ADVISORY

Maintenance decisions belong to a planner, not to a model, so the ceiling is advisory by design — the demo never schedules or shuts anything down on its own. To run it on a live fleet you replace the mock telemetry edge with a real MQTT/Sparkplug subscriber while preserving the typed signal contract and the advisory ceiling. The recommendation you verified against a deterministic trace arrives the same way from a live feed, and a human still owns the decision.